

Project 2: Fundamentals of Computer Vision

NON-LINEAR FILTERING: MEDIAN FILTER

1. What is the role of the kernel size in the Median filter?

Kernel size refers to the height and width of the region of pixels we examine for each computation. It must be a set of odd numbers greater than 1, (i.e. 3x3, 5x5, 7x7, etc.). In Median filtering, the kernel size relates to how large of a set of values we use to find the median value. Small 3x3 kernels can be good if your goal is image preservation, since they can preserve details and edges while still removing salt-and-pepper noise. Larger 5x5 kernels will remove more noise than the smaller kernel, and this can also mean a generalized blurring effect caused by finding the median of a larger number of values. Using a 7x7 kernel will result in the heaviest noise removal, but this also results in loss of edge clarity and many fine details.

- 3x3



- 5x5



- 7x7



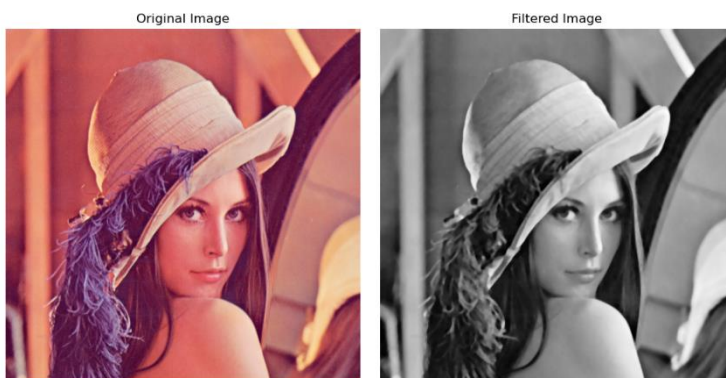
2. What is the significance of padding in the Median filter implementation?

Padding is important in Median filtering since it helps preserve the image's size from input to output. If you don't add padding to the image, then when the kernel is working through the edge pixels, it will not be able to center itself properly. As a result, details will be lost and the image will shrink. The median changes depending on what values we add to the edge in the form of padding. This is because the median value is mathematically defined as the number that separates the higher value half of the data set from the lower values. Zero padding can result in unnatural dark edges because zeros are added to the range used for finding the median, and zeros in a 0-255 scheme are valued as black. Reflect padding mirrors the image at the borders, which reduces these effects.

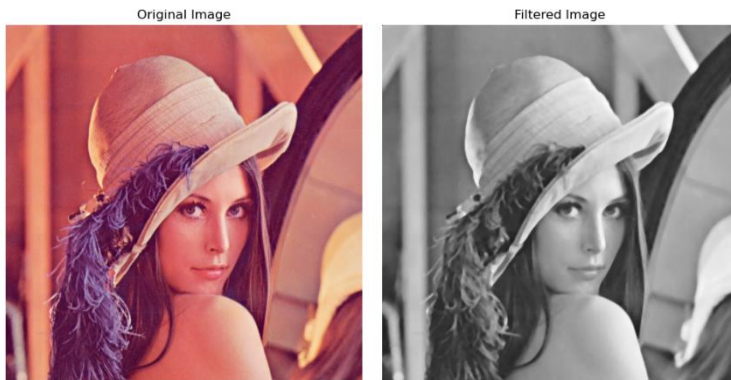
- *No Padding*



- *Reflect Padding*



- *Zero Padding*



3. How does the Median filter perform when used with an image that has salt-and-pepper noise?

Salt-and-Pepper noise are random, usually black-and-white pixels that appear scattered across the image. The Median filter works well to remove salt-and-pepper noise. To remove the most noise, it is better to use a larger kernel, as some imperfections will remain when using a small kernel. This is because the larger kernel includes more values in its median calculations so random salt-and-pepper noise is ignored to a greater degree. Median filtering naturally gets rid of most 0s and 255s since they are considered outliers, and this helps to remove random noise of types like salt-and-pepper. At higher density of salt-and-pepper pixels (> 30% frequency), we can start to see that Median filtering breaks down a bit because the noise starts to factor into the median calculations to a higher degree. Another way to say this is, Median filtering is basically using surrounding pixels to account for the ones that go missing with salt-and-pepper noise but at a certain point there are not enough non-corrupted pixels left to accurately restore the image. At 50% we can see random artifacts form because of this extremely high density of corrupted pixels. Using a larger kernel can help to reduce noise even with these high frequencies of salt-and-pepper noise, but small details are lost due to the extreme blurring effects of these sorts of kernels.

- *3x3 kernel with 10% Salt-and-Pepper*
- *7x7 kernel with 10% Salt-and-Pepper*
- *3x3 kernel with 50% Salt-and-Pepper*
- *7x7 kernel with 50% Salt-and-Pepper*

4. What advantages does the Median filter offer over other filters, such as Gaussian or average filters, when handling noisy images?

Since Median Filters are non-linear, they do not count the weighted sum of every pixel in a range the same way that linear (i.e. Gaussian & Average) filters do, and this makes them better suited for handling random impulse noise. Linear filters can be better for

uniform noise. Median filters work by looking at all the values in a given range that is being examined as a 1D list and finding the middle value; they exclude extremes like 0s and 255s the majority of the time. Because of the way they look at Median values, Median filters also have a higher rate of edge preservation and less blurring effects. For example, if there is a jump from 30 to 180 the Median filter has to choose one of those values, and unlike Gaussian or Average filters the result will not be some arbitrary mean value like 105, that falls between the two.

MORPHOLOGICAL OPERATIONS

6. Briefly describe the following key properties of morphological operations: translation invariance, monotonicity, idempotence, duality, and closure.

Morphological operations — such as **erosion**, **dilation**, **opening**, **closing**, and **gradient** — have several theoretical properties that describe how they behave on binary images and shapes.

- **Translation Invariance**

A morphological operation produces the same result even if the image is shifted. If an object moves in the image, the output of erosion or dilation moves by the same amount, without any change in shape or intensity.

- **Monotonicity**

If image A is contained in image B ($A \subseteq B$), then erosion and dilation will preserve this inclusion. In other words, $erosion(A) \subseteq erosion(B)$ and $dilation(A) \subseteq dilation(B)$. This means that larger or brighter regions will always produce equal or larger morphological results.

- **Idempotence**

Once an image has been eroded or dilated, repeating the same operation with the same kernel does not further change the result. After one pass, the operation stabilizes:

$$\varepsilon(\varepsilon(A)) = \varepsilon(A), \quad \delta(\delta(A)) = \delta(A)$$

- **Duality**

Erosion and dilation are duals of each other with respect to image complement and kernel reflection:

This means that the complement of a dilation is equivalent to an erosion on the complement image, and vice versa.

- **Closure**

Closure is the combination of **dilation followed by erosion**, using the same kernel. It fills small holes and connects close white regions while preserving the overall shape of the objects. Formally:

7. Provide an example of the closure property by executing your code for dilation and erosion. Justify your conclusion.

Objective:

To verify the **closure property**, which states that applying dilation followed by erosion using the same structuring element fills small holes and connects nearby foreground regions while maintaining the overall shape.

Implementation:

```
def erosion(image, kernel):
    h, w = image.shape
    kh, kw = kernel.shape
    ph, pw = kh//2, kw//2
    padded = np.pad(image, ((ph, ph), (pw, pw)), mode='constant', constant_values=0)
    out = np.zeros_like(image, dtype=np.uint8)
    for i in range(h):
        for j in range(w):
            region = padded[i:i+kh, j:j+kw]
            if np.all(region[kernel==1] == 255):
                out[i, j] = 255
    return out
```

To perform **closing**, you simply define:

and call it with your same rectangular kernel:

Results:

After applying **closing**, small black holes inside the white regions are filled, and nearby white objects become connected.

The outer boundaries remain mostly unchanged, proving that the operation smooths and unifies regions while maintaining the object's shape.

Justification:

The closure property combines dilation followed by erosion:

- **Dilation** expands the white foreground areas, closing gaps and holes.
- **Erosion** then trims back the boundaries, restoring the original contour size while keeping the filled gaps closed.

If the operation is applied twice with the same kernel:

The result remains unchanged, confirming that **closure is idempotent** and stabilizes after the first application.

8. Provide an example of the idempotence property by executing your code for erosion. Justify your conclusion.

Objective:

To verify the **idempotence property** of erosion, which states that once an image has been eroded, applying erosion again with the same structuring element does not change the result any further.

Implementation:

I implemented erosion as:

```
def erosion(image, kernel):
    h, w = image.shape
    kh, kw = kernel.shape
    ph, pw = kh//2, kw//2
    padded = np.pad(image, ((ph, ph), (pw, pw)), mode='constant', constant_values=0)
    out = np.zeros_like(image, dtype=np.uint8)
    for i in range(h):
        for j in range(w):
            region = padded[i:i+kh, j:j+kw]
            if np.all(region[kernel==1] == 255):
                out[i, j] = 255
    return out
```

Then I used my 3×3 rectangular structuring element:

and tested the property by applying erosion twice:

Results:

After the first erosion, the boundaries of the white regions shrink and small isolated white points are removed.

When erosion is applied a second time, the image remains unchanged, demonstrating that further erosion no longer affects the result.

Justification:

The idempotence property of erosion is mathematically expressed as:

This means that once all pixels that do not fully fit the structuring element have been removed, repeating the operation has no additional effect.

Your pixel-by-pixel implementation confirms this — after the first erosion, all remaining white pixels are fully surrounded by other white pixels matching the kernel, so no further pixels can be removed.

Conclusion:

The second erosion produced the same image as the first:

This demonstrates that your erosion implementation is **idempotent** and that morphological erosion stabilizes after a single application with a fixed structuring element.

WATERMARKING

9. Describe the advantages and disadvantages of implementing watermarking using a combination of linear filtering, non-linear filtering, and morphological operations. At most use a maximum of 10 lines.

Advantages:

- Gaussian smoothing reduces watermark noise, avoiding grainy blend artifacts.
- Edge enhancement (Sobel/Laplacian) strengthens contours, keeping text/logo readable at low alpha.
- Morphology (dilation/erosion) restores thin strokes and normalizes line thickness.
- Alpha blending gives fine control over visibility and a natural integration.

Disadvantages:

- Sensitive to hyperparameters (kernels, iterations, alpha); small changes can cause halos or over-smoothing.
- Possible loss of fine details after heavy smoothing or erosion.
- Higher computational cost and complexity than a raw `addWeighted` overlay.
- On highly textured backgrounds, dilation may introduce visible artifacts.

10. Choose one advantage of your implementation and illustrate it with an example from the output of your code.

Advantage: Improved readability at low transparency due to contour strengthening.

In the **Processed Watermark** panel (after Gaussian → Sobel → dilation/erosion → light blur), contours appear sharper.

With **alpha = 0.3**, the **Watermarked Image** stays clear over textured regions (Lenna's hat) without washing out background colors.